

A Comparative Study of Jajuk and JHotDraw 5.2

Tomás Alexandre

73213

tf.alexandre@campus.fct.unl.pt

NOVA School of Science and Technology
Portugal

Nicolae Iachimovschi

73391

n.iachimovschi@campus.fct.unl.pt

NOVA School of Science and Technology
Portugal

ABSTRACT

This report presents a reverse engineering analysis of two Java-based systems, Jajuk 11.0 and JHotDraw 5.2, using tools such as inFusion, MetricsTree, and SonarQube for IDE. The analysis begins with the Overview Pyramid and progressively focuses on specific aspects of the systems, including metrics, code smells, design disharmonies, and code duplication. The two systems reveal different quality profiles, with Jajuk displaying significant structural problems accumulated over the years of development, while JHotDraw 5.2 demonstrates a more disciplined object-oriented design.

1 SYSTEM DESCRIPTION

1.1 Jajuk

Jajuk¹ (v11.0, “Deepest Blue”, released November 2019) is an open-source desktop media player for managing local music libraries and web radio, available for Windows, Linux, and macOS. It is a Java application built with Apache Ant, requiring JRE 1.8+, with 1080 Java source files in total.

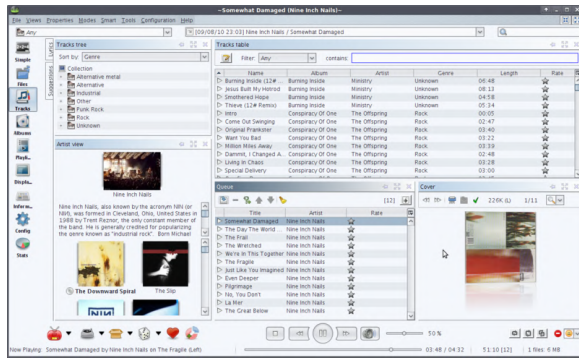


Figure 1: Jajuk Interface

Since the project uses Ant rather than Maven or Gradle, no automatic import was available in IntelliJ. The folder `src/main/java` was manually configured as Sources Root, `src/test/java` as Test Sources Root, and `src/main/resources` as Resources Root. The `lib/` folder was added as a module dependency to resolve previously invalid imports, and the SDK was set to Microsoft OpenJDK 17, with residual errors remaining after setup, as expected given its legacy nature, which do not affect the static analysis.

¹<https://github.com/jajuk-team/jajuk>

1.2 JHotDraw 5.2

JHotDraw 5.2² is an open-source Java drawing framework designed to serve as a foundation for structured drawing editors. Unlike Jajuk, it is not a standalone application, but a reusable framework intended to be extended by developers, containing 186 classes in 11 packages.

For the IntelliJ setup, the `src/` folder was configured as Sources Root and JDK 11 was selected to ensure better compatibility with the legacy codebase, with some residual errors remaining after setup, as expected, which do not affect the static analysis.

2 ANALYSIS – JAJUK

2.1 Overview

The analysis of Jajuk begins with the Overview Pyramid generated by inFusion (Figure 2), which provides a high-level structural view of the system. Jajuk is a large system, with 564 classes across 41 packages, 3 468 methods, and 47 438 lines of code.

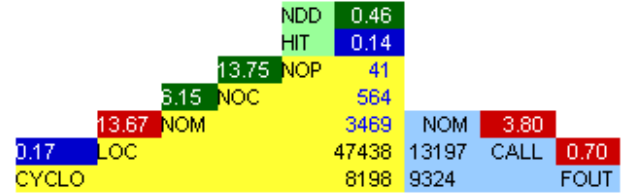


Figure 2: Jajuk Overview Pyramid

Looking at the right side of the pyramid, we can observe that `FOUT/CALL = 0.70` is highlighted in red, meaning that 70% of all method calls target a different class, indicating high coupling dispersion across the system. We can also see that `NOM/NOC = 3.15` suggests that classes contain few methods on average, though as we will see in the following sections, this hides a highly unbalanced distribution. As for inheritance, `HIT = 0.14` and `NDD = 0.46` confirm that class hierarchies tend to be shallow and of average width.

Classes	Methods	Packages
God Class 27	Feature Envoy 119	Cyclic Dependencies 35
Data Class 24	Brain Method 64	Unstable Dependencies 8
Brain Class 4	Intensive Coupling 27	
Refused Parent Bequest 1	Dispersed Coupling 28	
Tradition Breaker 5	Shotgun Surgery 73	

Figure 3: Code Disharmonies detected in Jajuk

²<https://sourceforge.net/projects/jhotdraw/>

The disharmonies reported by inFusion (Figure 3) reveal a problematic picture: 27 God Classes and 24 Data Classes indicate both over-responsible classes and a lack of encapsulation, while 4 Brain Classes point to concentrated complexity. At method level, 119 Feature Envy and 73 Shotgun Surgery cases suggest that changes tend to spread across many classes, making future evolution costly. The 35 cyclic and 8 unstable package dependencies further confirm that Jajuk’s architectural problems are deep.

2.2 Size and Complexity

To better understand the root causes behind these numbers, we examine size and complexity in more detail. The NOM Distribution Chart from MetricsTree (Figure 4) reveals that 36.7% of classes surpass the regular range, with 20.2% classified as high, 8.4% as very-high, and 8.1% as extreme. The treemap in Figure 4 confirms this visually, with a concentration of orange and red nodes in the upper area contrasting with the green nodes that dominate the lower right, revealing a highly uneven size distribution across the system.

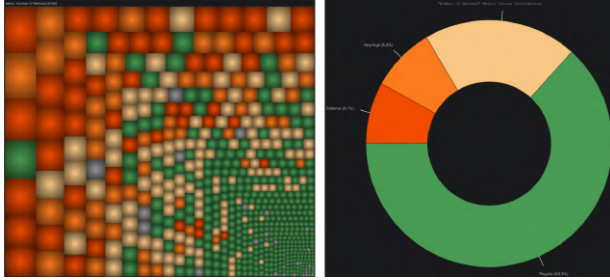


Figure 4: Treemap and Distribution Chart of Classes based on NOM in Jajuk

Looking at complexity, the WMC distribution (Figure 5) tells a more concerning story. According to the Metrics Distribution Chart, only 59.1% of classes fall within the regular range, while 40.9% surpass it with 25.4% classified as high, 5.2% as very-high, and 10.3% as extreme.

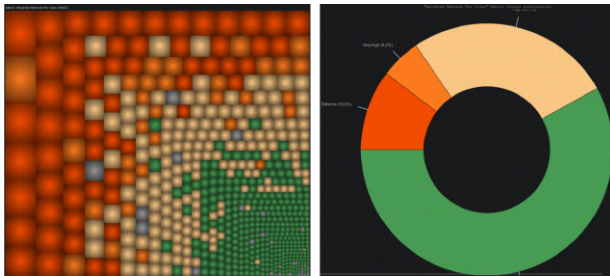


Figure 5: Treemap and Distribution Chart of Classes based on WMC in Jajuk

Compared to the NOM distribution, the proportion of extreme WMC values is notably higher, suggesting that the problem is not just the number of methods but their internal complexity. The

treemap in Figure 5 makes this particularly visible: a dense concentration of orange and red nodes dominates the upper left corner, with complexity decreasing progressively towards the lower right.

Table 1: Top 5 classes by WMC in Jajuk

Class	WMC	NOM	CCC	NCSS
CoverView	229	35	197	776
QueueModel	223	59	187	616
IconLoader	213	3	216	636
Device	198	45	161	517
UtilSystem	159	49	117	501

The Table 1 lists the five classes with the highest WMC values. The most extreme case is CoverView (WMC = 229), which accumulates responsibilities across UI layout, remote cover, downloading, image processing, and web search query construction.

It is worth noting that CoverView is not an isolated case. As shown in Table 1, the remaining four classes display similar concerning values, and the 27 God Classes identified by inFusion indicate that this pattern of accumulated responsibilities is common across the system.

2.3 Coupling

Moving on to coupling, the picture is even more concerning. As shown in Figure 6, the CBO treemap is dominated by orange and red nodes across almost the entire canvas, with only a small set of green nodes visible in the lower right corner.

This contrasts with the WMC treemap observed earlier, where a substantial green area was still present, as high coupling dominates the visualization and affects most classes in the system.

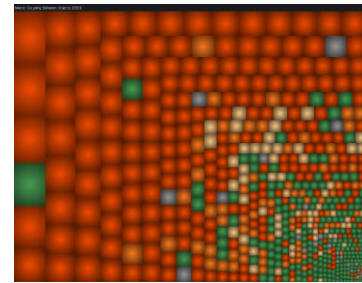


Figure 6: Treemap of Classes based on CBO in Jajuk

This is consistent with FOUT/CALL = 0.70 highlighted in the Overview Pyramid, and is further confirmed by the CBO distribution in Figure 7: the box plots show that across all fitness function groups, the range sits consistently between 25 and 70, with numerous outliers exceeding 100, well above the regular range of [0..14] [3].

Among the most extreme cases, as shown in Table 2, CoverView reaches CBO = 100 and RFC = 197, while UtilGUI presents CBO = 101 with a TCC of 0.00, indicating zero cohesion despite its high coupling.

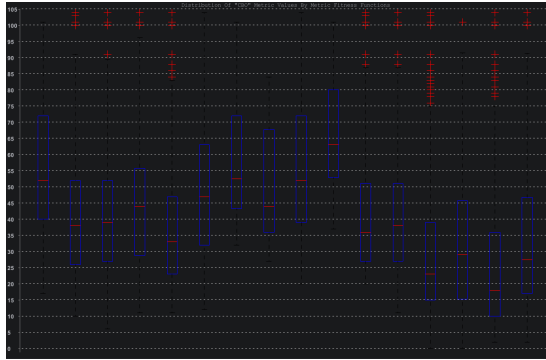


Figure 7: Distribution of CBO Classes in Jajuk

Table 2: Top 5 classes by CBO in Jajuk

Class	CBO	RFC	ATFD	TCC
AbstractTableView	104	133	6	0.52
ActionManager	103	106	0	0.00
UtilGUI	101	171	7	0.00
FilesTreeView	101	126	5	0.03
CoverView	100	197	12	0.33

Classes with such high CBO values tend to become fragile centers, as changes in the classes on which they are based can easily spread and cause failures, making the future evolution of the system particularly risky.

2.4 Inheritance

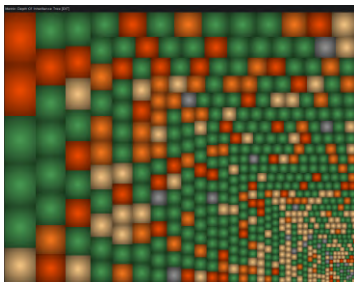


Figure 8: Treemap of Classes based on DIT in Jajuk

Regarding inheritance, the Overview Pyramid already indicates relatively shallow hierarchies, with HIT = 0.14 and NDD = 0.46. This observation is reinforced by the DIT treemap in Figure 8, where a large portion of the visualization is green, corresponding to classes with low inheritance depth, while a smaller set of orange and red nodes highlights the presence of some deeper hierarchies.

Overall, these results suggest that inheritance is not extensively used as a structural mechanism in Jajuk. Instead, the system appears to concentrate behavior within individual classes, a pattern consistent with the presence of the God Classes and Brain Classes identified earlier.

2.5 Duplication

SonarQube reports 271 duplication issues across 126 files, highlighting a significant level of code redundancy. As illustrated in Figure 9, these duplications are not limited to specific areas but are widespread, affecting classes across all packages in the system.

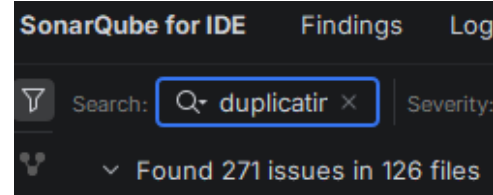


Figure 9: SonarQube Duplication Issues in Jajuk

Looking more closely at a specific example, Figure 10 shows the duplication issues detected in CoverView.java, a class already identified as the most complex in the system. SonarQube flags two repeated string literals: "[grow]", a layout descriptor appearing 3 times, and "<html>", an HTML tag appearing 3 times.

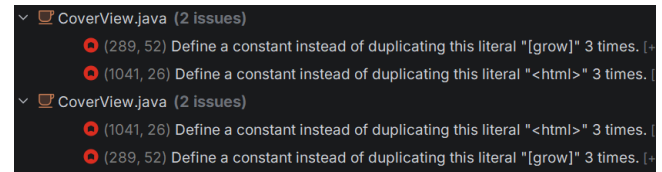


Figure 10: SonarQube Duplication Issues in CoverView.java

Although these may seem minor in isolation, the fact that the same pattern repeats across 126 files reveals a systemic absence of constant extraction throughout the codebase, increasing the risk of inconsistencies whenever these values need to change.

3 ANALYSIS – JHOTDRAW 5.2

3.1 Overview

JHotDraw 5.2 is a considerably smaller system than Jajuk, with 186 classes across 11 packages, 1 390 methods, and 7 712 lines of code. The Overview Pyramid in Figure 11 provides a high-level structural view of the system.

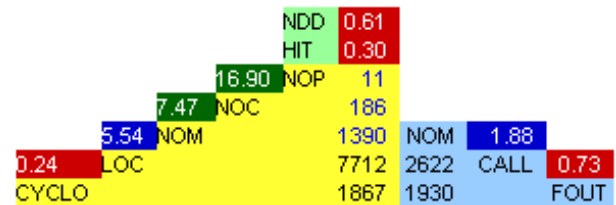


Figure 11: JHotDraw 5.2 Overview Pyramid

On the right side of the pyramid, FOUT/CALL = 0.73 is highlighted in red, indicating a coupling dispersion comparable to Jajuk's 0.70, and particularly concerning given the system's much

smaller scale. $NOM/NOC = 7.47$ is notably higher than Jajuk's 3.15, suggesting that classes carry more methods on average. Regarding inheritance, $HIT = 0.30$ and $NDD = 0.61$ reflect deeper and wider hierarchies than Jajuk (0.14 and 0.46), consistent with JHotDraw's nature as a reusable object-oriented framework designed around extensibility.

Classes		Methods	
Data Class	2	Feature Envy	80
Refused Parent Bequest	2	Brain Method	1
Packages		Intensive Coupling	1
Unstable Dependencies	1	Shotgun Surgery	9

Figure 12: Code Disharmonies detected in JHotDraw 5.2

The disharmony profile in Figure 12 contrasts sharply with Jajuk's: no God Classes are detected, compared to the 27 found in Jajuk, and only 2 Data Classes versus 24, suggesting considerably better encapsulation. The 80 Feature Envy methods stand out as the most prevalent issue, while 9 Shotgun Surgery cases and a single Brain Class indicate limited complexity. At package level, only 1 Unstable Dependency is detected, compared to 8 in Jajuk, reflecting a more stable architectural structure.

3.2 Size and Complexity

The NOM distribution from MetricsTree reveals that 43.9% of classes surpass the regular range, with 32.4% classified as high, 4.1% as very-high, and the remainder as extreme. Compared to Jajuk's 36.7%, this is a notably higher proportion, consistent with the elevated $NOM/NOC = 7.47$ observed in the pyramid. The treemap in Figure 13 confirms this pattern: while green nodes are visible in the lower right, the upper left is heavily dominated by red and orange nodes, indicating that a significant portion of classes carry a disproportionate number of methods.

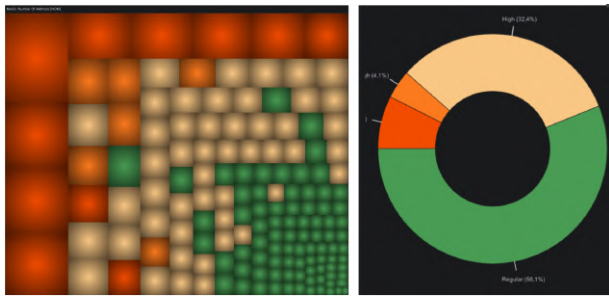


Figure 13: Treemap and Distribution Chart of Classes based on NOM in JHotDraw 5.2

Looking at complexity, the WMC distribution tells a more balanced story: 62.2% of classes fall within the regular range, while 37.8% surpass it, with the high category dominating and only 2.0% classified as very-high and 6.8% as extreme. Compared to Jajuk's 40.9%, the proportion of classes outside the regular range is slightly lower, and the extreme category is notably smaller (6.8% vs 10.3%),

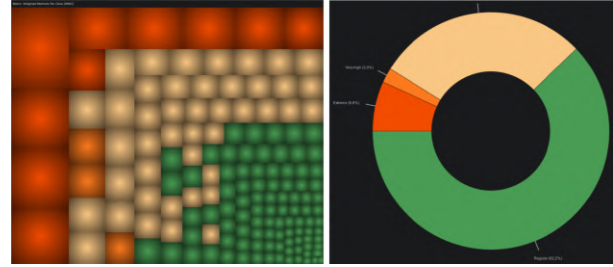


Figure 14: Treemap and Distribution Chart of Classes based on WMC in JHotDraw 5.2

suggesting that while problematic classes exist, their internal complexity is less severe.

The WMC treemap in Figure 14 reflects this: the upper left remains dominated by red and orange nodes, but the transition to green is more gradual than in Jajuk, with a larger green area visible in the lower right.

Table 3: Top 5 classes by WMC in JHotDraw

Class	WMC	NOM	CCC	NCSS
StandardDrawingView	108	61	45	278
DrawApplication	81	59	28	376
PolygonFigure	72	34	38	277
CompositeFigure	71	33	38	165
DrawApplet	61	34	31	262

Table 3 lists the five classes with the highest WMC values in JHotDraw 5.2. The most complex class is StandardDrawingView (WMC = 108), which centralizes rendering and interaction responsibilities for the drawing canvas.

Notably, even the highest WMC value in JHotDraw (108) is less than half of Jajuk's most extreme case (229), suggesting that complexity, while present, is considerably more contained. The remaining four classes show a gradual decrease in WMC, with no single class dominating the complexity profile to the same degree observed in Jajuk.

3.3 Coupling

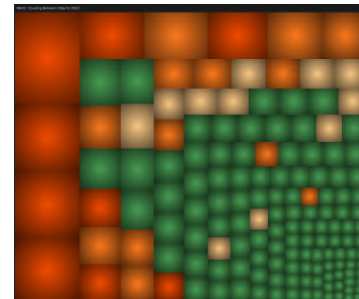


Figure 15: Treemap of Classes based on CBO in JHotDraw 5.2

The CBO treemap in Figure 15 presents a significantly different picture from Jajuk. While red and orange nodes still dominate the upper left, a large green area is clearly visible in the lower right, indicating that a significant portion of classes maintains low coupling levels. This differs significantly with Jajuk's almost entirely red canvas, suggesting that coupling in JHotDraw, while present, is more concentrated.

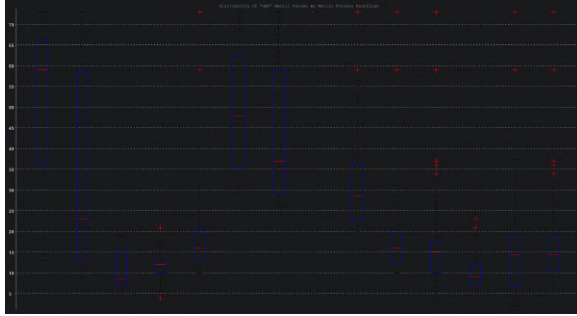


Figure 16: Distribution of CBO Classes in JHotDraw 5.2

This is consistent with $FOUT/CALL = 0.73$ from the Overview Pyramid, and is further confirmed by the CBO distribution in Figure 16: the box plots show considerably lower interquartile ranges than Jajuk, sitting mostly between 10 and 65, with outliers reaching around 75, well below Jajuk's outliers exceeding 100 [3].

Table 4: Top 5 classes by CBO in JHotDraw 5.2

Class	CBO	RFC	ATFD	TCC
DrawApplication	73	167	12	0.0081
DrawApplet	59	113	7	0.1373
StandardDrawingView	37	128	10	0.0327
JavaDrawApp	36	53	1	0.0152
AbstractFigure	34	63	2	0.0369

Among the most extreme cases, Table 4 shows that the class `DrawApplication` reaches the highest CBO = 73 with RFC = 167 and a very low TCC of 0.0081, indicating high coupling combined with near-zero cohesion. `StandardDrawingView`, already identified as the most complex class in the system, also appears here with CBO = 37 and RFC = 128, reinforcing its role as a central and problematic element.

Importantly, even the highest CBO value in JHotDraw (73) falls well below Jajuk's most coupled class (104), confirming that while coupling issues exist, they are considerably more contained.

3.4 Inheritance

Regarding inheritance, the Overview Pyramid already indicated deeper and wider hierarchies than Jajuk, with HIT = 0.30 and NDD = 0.61 compared to 0.14 and 0.46. The DIT treemap in Figure 17 confirms this: unlike Jajuk's predominantly green canvas, orange and red nodes are more widely distributed, reflecting a more active use of inheritance as a structural mechanism.

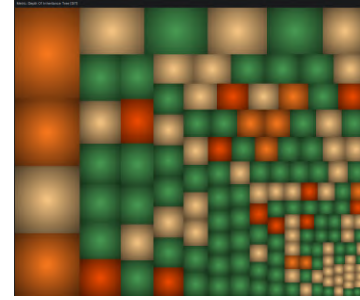


Figure 17: Treemap of Classes based on DIT in JHotDraw 5.2

This is consistent with JHotDraw's nature as a reusable framework designed to be extended, where deep hierarchies are an intentional design choice rather than a sign of accumulated complexity. The absence of God Classes and the low number of Data Classes further support this interpretation, suggesting that behaviour is properly distributed across the hierarchy rather than concentrated in isolated classes.

3.5 Duplication

SonarQube reports 7 duplication issues across 2 files, a clear contrast to the 271 issues found across 126 files in Jajuk, suggesting a considerably more disciplined approach to code reuse.

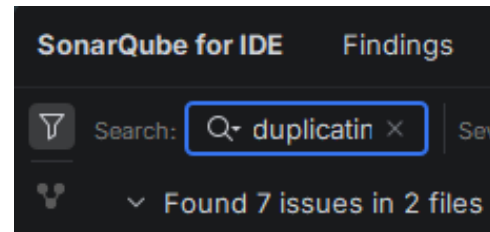


Figure 18: SonarQube Duplication Issues in JHotDraw 5.2

Looking more closely at the specific cases, Figure 19 shows that `DrawApplet.java` is responsible for 5 of the 7 issues, with repeated string literals such as "FillColor" appearing 3 times, "ArrowMode" appearing 6 times, and "TextColor" appearing 3 times. `DrawApplication.java` presents 2 additional issues, with "ArrowMode" repeated 4 times and "FontStyle" repeated 3 times.

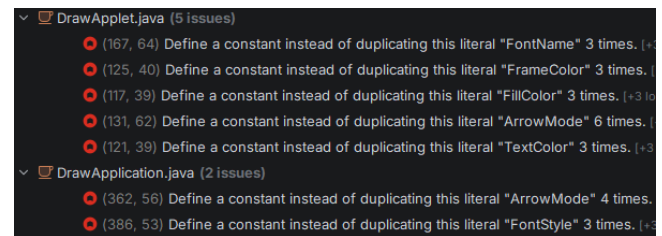


Figure 19: SonarQube Duplication Issues in `DrawApplet.java` and `DrawApplication.java`

4 CONCLUSION

The two systems reveal markedly different quality profiles, as summarised in Table 5.

Table 5: Comparative Summary of Jajuk and JHotDraw 5.2

Metric / Indicator	Jajuk	JHotDraw 5.2
Classes	564	186
Packages	41	11
Methods	3 468	1 390
Lines of Code	47 438	7 712
NOM/NOC	3.15	7.47
FOUT/CALL	0.70	0.73
HIT	0.14	0.30
NDD	0.46	0.61
WMC outside regular (%)	40.9	37.8
NOM outside regular (%)	36.7	43.9
Max WMC	229	108
Max CBO	104	73
God Classes	27	0
Data Classes	24	2
Brain Classes	4	1
Feature Envy	119	80
Shotgun Surgery	73	9
Cyclic Dependencies	35	0
Unstable Dependencies	8	1
Duplication Issues	271	7
Files with Duplication	126	2

Jajuk, a large application with 564 classes and 47 438 lines of code, accumulates significant structural problems: 27 God Classes, 24 Data Classes, and 4 Brain Classes indicate a widespread lack of encapsulation and concentrated complexity. CBO values consistently exceeding 100 and 271 duplication issues across 126 files suggest that future evolution will be expensive and susceptible to errors. The system's lack of shallow inheritance hierarchies further reinforce the picture of a codebase that grew without sufficient architectural discipline.

JHotDraw 5.2, despite being considerably smaller with 186 classes, presents a more disciplined object-oriented design. The complete absence of God Classes, only 2 Data Classes, and duplication only in 2 files reflect a cleaner separation of responsibilities. Its deeper inheritance hierarchies (HIT = 0.30 vs 0.14) are consistent with its nature as a reusable framework designed around extensibility, where inheritance is an intentional design choice. The main concern lies in the 80 Feature Envy methods and the high NOM/NOC = 7.47, suggesting that some classes carry too many methods and that inter-class data access patterns deserve attention in future refactoring efforts.

In summary, JHotDraw 5.2 demonstrates stronger object-oriented design principles, while Jajuk's accumulated technical debt poses a significant risk to its long-term maintainability.

References

- [1] Jajuk Team. *Jajuk - Advanced Jukebox Software*. [Online]. Available: <https://jajuk.info/> [Accessed: Mar. 2026].
- [2] JHotDraw Team. *JHotDraw 5.2: Framework for Technical and Structured Graphics*. [Online]. Available: <https://sourceforge.net/projects/jhotdraw/> [Accessed: Mar. 2026].
- [3] M. Lanza, R. Marinescu. *Object-Oriented Metrics in Practice*. Springer, 2006.